JAVA UNIT-2 MATERIAL

0Ch. No.	Topic	Detail+s	Marks
2	Inheritance and java packages	• Universal Class (Object Class) • Access Specifiers (public, private, protected, default, private protected) • Constructors in inheritance • Method Overriding • Interface, Object Cloning • Nested and Inner Class • Abstract and -Final Class • Normal import and Static Import • Introduction to Java API Packages and imp. Classes java.lang , java.util ,java.io , java.net, java.awt ,java.awt.event , java.applet , java.swing • java.lang Package Classes (Math, Wrapper Classes, String, String Buffer) • java.util Package Classes (Random, Date, GregorianCalendar • StringTokenizer, Collection in Java -Vector, HashTable, LinkedList, SortedSet, Stack, Queue, Map • Creating and Using UserDefined package and sub-package	14

Q-1 Explain Universal Class (Object Class in Java)

👉 According to your PDF

✅ **What is Universal Class?**

- The **Object class** is the **parent of all classes in Java**.
- It is the **top (root) class** in Java.

👉 This means:

- Every class in Java **automatically uses Object class**.
- Even if you don't write it, Java adds it by default.

✅ **Important Points**

- Object class is in **java.lang package**.
- Every class:
  - **Direct child** → if it does not extend any class
  - **Indirect child** → if it extends another class
- So, **all classes get Object class methods**.

👉 Because of this:

- Object class is called the **root of inheritance**.

✅ **Why Object Class is useful?**

- It helps when you **don't know the type of object**.
- You can still work with any object using Object class.

✅ **Important Methods of Object Class**

1. toString()

- Converts object into string
  👉 Example: printing object

---

2. hashCode()

- Gives a unique number for object
  👉 Used in collections (like searching)

---

3. equals()

- Compares two objects
  👉 Checks if both are same or not

---

4. getClass()

- Returns the class name of object
  👉 Helps to know object type at runtime

---

5. finalize()

- Called before object is deleted
  👉 Used in garbage collection
  👉 Runs only once

---

6. clone()

- Creates a copy of object
  👉 New object = same as old object

---

7. notify()

- Wakes up one waiting thread
- એક રાહ જોઈ રહેલા થ્રેડને જગાડે છે

---

8. notifyAll()

- Wakes up all waiting threads
- બધા રાહ જોઈ રહેલા થ્રેડોને જાગૃત કરે છે

---

9. wait()

- Makes thread wait until notification
- થ્રેડને સૂચના સુધી રાહ જોવા માટે બનાવે છે

✅ Simple Example

```
class Test {
  public static void main(String[] args) {
    String s = "Hello";
    System.out.println(s.toString()); // from Object class
  }
}
```

✅ Short Summary

- Object class = **parent of all classes**
- Present in **java.lang**
- Provides common methods like:
  - toString()
  - equals()
  - hashCode()
- It is the **base of inheritance in Java**

OR

🔷 What is Universal Class (Object Class)?

👉 Simple idea:

- In Java, **every class is connected to one main class**
- That main class is called **Object class**

👉 So:

Object class = father of all classes

🔷 Real Life Example

Think like this 👇

- 👨 Father (Object class)
- 👦 Child 1 (Student class)
- 👧 Child 2 (Employee class)

👉 All children get things from father
👉 Same in Java:

- All classes get features from **Object class**

🔷 **Very Simple Definition**

👉 **Object class is the top class in Java**
👉 All classes **automatically use it**

🔷 **Important Points (Very Easy)**

- You don't need to write it
- Java **automatically adds Object class**
- All classes can use its methods

🔷 **Example (Very Easy)**

```
class A {
}

class B extends A {
}
```

👉 Here:

- A is child of Object
- B is also child of Object (indirectly)

🔷 **Why it is Important?**

👉 Because it gives **common functions** to all classes

🔷 Easy Methods (Remember only basic)

1. toString()

👉 Converts object to text

2. equals()

👉 Compare two objects

3. hashCode()

👉 Gives unique number

4. getClass()

👉 Tells class name

🔷 One Line Memory Trick

👉 **“Object class = root (starting point) of all classes”**

🔷 Short Exam Answer (Write this)

👉 The Object class is the parent class of all classes in Java.
👉 It is present in java.lang package.
👉 All classes directly or indirectly inherit it.
👉 It provides common methods like toString(), equals(), hashCode().

MCQ		
Sr No.	Question	Answer
1	Which is the universal class in java?	Object
2	Object class is present in .....package	java.lang
3	Which method of object class is used to return a hash value that is used to search the object in collection?	hashcode()
4	Which method of object class generates exactly same object as this object?	clone()
5	Which method of object class is called just before the object is garbage collected?	finalize()

✅ **Q-2 Access Specifier (Simple Explanation)**

🔷 **What is Access Specifier?**

👉 **Access Specifier = rules of visibility**

👉 It decides:

- Who can use variable, method, class

👉 Simple:

It controls "who can access your code"

🔷 **Example (Real Life)**

Think like this 👇

- 🔒 Private → Only you can use
- 🏠 Default → Only your house people
- 👨‍👩‍👧 Protected → Family + relatives
- 🌍 Public → Everyone can use

🔷 **Types of Access Specifiers**

1️⃣ Private

👉 Only inside same class

❌ Cannot use outside class

✔ **Example:**

```
class A {
  private int data = 40;

  private void msg() {
    System.out.println("Hello");
  }
}
```

👉 Outside class → ❌ Error

2️⃣ Default (No keyword)

👉 If you don't write anything → default

👉 Accessible:

- Only inside same package

✔ **Example:**

```
class A {
  void display() {
    System.out.println("Hello");
  }
}
```

👉 Works inside same package ✅
👉 Outside package ❌

3️⃣ Protected

👉 Accessible:

- Same package ✅

- Subclass (child class) ✅ In different package → only subclass (using extends) can access

✔ **Example:**

```
class A {
  protected void display() {
    System.out.println("Hello");
  }
}

class B extends A {
}
```

👉 B can use display() ✅

4️⃣ Public

👉 Accessible from anywhere

👉 No restriction

✔ **Example:**

```
class A {
  public void display() {
    System.out.println("Hello");
  }
}
```

👉 Any class can use it ✅

🔷 **Easy Table (Remember this)**

Type	Same Class	Same Package	Subclass	Anywhere
Private	✅	❌	❌	❌
Default	✅	✅	❌	❌
Protected	✅	✅	✅	❌
Public	✅	✅	✅	✅

🔷 **Short Exam Answer**

👉 Access specifiers are used to control the accessibility of class members.
👉 There are 4 types: Private, Default, Protected, Public.
👉 Private → only inside class
👉 Default → same package
👉 Protected → same package + subclass
👉 Public → accessible everywhere

🔷 **One Line Trick (Very Important)**

👉
Private < Default < Protected < Public

(Accessibility increases →)

MCQ		
Sr No.	Question	Answer
1	.....modifier is accessible from everywhere	public
2	The access level of .....modifier is within the package and outside the package through inheritance	protected
3	Which is the default access modifier?	default

👉 *Below Question is Most Important for Exam Purpose*

👉 default and protected that is create confusion

Got it 👍 This confusion is **very common**. I will explain **Default vs Protected** in the easiest way.

🔥 **First Understand ONE KEY IDEA**

👉 Everything depends on **2 things only**:

1. Same package
2. Different package

🟡 **What is "Package"? (Very Simple)**

👉 Package = folder of classes

Example:

package p1 → Class A, Class B
package p2 → Class C

🟢 **1. DEFAULT (No keyword)**

👉 Rule:

Only works **inside same package**

❌ Outside package → NOT allowed
❌ Subclass in different package → NOT allowed

✔ **Example**

```
package p1;

class A {
  void show() {
    System.out.println("Hello");
  }
}
```

👉 Same package → works ✅
👉 Different package → error ❌

🔵 **2. PROTECTED**

👉 Rule:

Works in:

- Same package ✅
- Subclass (even in different package) ✅

✓ Example

```
package p1;

class A {
  protected void show() {
    System.out.println("Hello");
  }
}
```

```
package p2;

class B extends A {
  public static void main(String[] args) {
    B obj = new B();
    obj.show(); // works ✅
  }
}
```

👉 Because B is **child class (extends A)**

🚀 MAIN DIFFERENCE (VERY IMPORTANT)

Feature	Default	Protected
Same package	✅	✅
Different package	❌	❌
Subclass (different package)	❌	✅

👉 "In default, subclass is available in same package, so does default work?" -> yes or no

✅ Q-3 Constructor in Inheritance (Simple Explanation)

🔷 What is Constructor?

👉 Constructor = special method
👉 Same name as class
👉 No return type
👉 Runs **automatically when object is created**

✔ Example:

```
class A {
  A() {
    System.out.println("Constructor called");
  }
}
```

🔷 Important Points

- Constructor name = class name
- No return type ❌
- Runs when object is created ✅
- Cannot be:
  - static ❌
  - abstract ❌
  - final ❌
- Cannot be overridden ❌

🔥 Main Concept: Constructor in Inheritance

👉 VERY IMPORTANT RULE:

Parent class constructor runs FIRST
Then child class constructor runs

🔷 **Why?**

👉 Because child class depends on parent
👉 So Java first initializes parent

🟢 **Example (Your Code Simplified)**

```
class Parent {
  Parent() {
    System.out.println("Parent constructor");
  }
}

class Child extends Parent {
  Child() {
    System.out.println("Child constructor");
  }
}

class Child1 extends Child {
  Child1() {
    System.out.println("Child1 constructor");
  }
}

public class Test {
  public static void main(String[] args) {
    Child1 obj = new Child1();
  }
}
```

🔥 **Output (IMPORTANT)**

```
Parent constructor
Child constructor
Child1 constructor
```

🧠 Easy Understanding (Real Life)

Think like this 👇

👴 Grandfather → 👨 Father → 🧑 Child

👉 When child is born:

1. Grandfather comes first
2. Then Father
3. Then Child

🔷 Step-by-Step Flow

When this runs:

Child1 obj = new Child1();

👉 Java does:

1. Call Parent constructor
2. Then Child constructor
3. Then Child1 constructor

🎯 One Line Rule (VERY IMPORTANT)

👉
Constructor runs from top to bottom (Parent → Child)

✅ Short Exam Answer

👉 A constructor is a special method with same name as class and no return type.
👉 It is automatically called when object is created.
👉 In inheritance, parent class constructor is executed first, followed by child class constructor.
👉 This ensures proper initialization of objects.

🧠 Memory Trick

👉
“Parent first, Child next”

MCQ		
Sr No.	Question	Answer
1	In case of constructor in inheritance, the order of execution of constructor will be	parent class constructor child class constructor
2	In case of constructor in multilevel inheritance, the order of execution of constructor will be.....	parent class constructor child class constructor child1 constructor

✅ **Q-4 Interface (Simple Explanation)**

🔷 **What is Interface?**

👉 **Interface = blueprint (design) of a class**

👉 Simple:

It only tells **WHAT to do**, not **HOW to do**

🔷 **Real Life Example**

Think 👇

📝 Exam paper:

- It tells questions (what to write)
- But **you write answers (how to do)**

👉 Same:

- Interface → gives methods (no body)
- Class → writes code (body)

🔷 **Important Points (Very Easy)**

- Interface has:
  - Only **method names (no body)**
- Used for:
  - **Abstraction** (hide details)
  - **Multiple inheritance**

🔷 **Syntax**

```
interface A {
  void show(); // no body
}
```

🔷 **How to Use Interface?**

👉 Use keyword: **implements**

✔ **Example:**

```
interface A {
  void show();
}

class B implements A {
  public void show() {
    System.out.println("Hello");
  }
}
```

👉 Class B MUST write method body ✅

🔥 **Multiple Inheritance using Interface**

👉 Java does NOT allow:
❌ class A, B → class C (not allowed)

👉 But using interface:
✅ possible

✓ **Example (Your code simplified)**

```
interface A {
  void display();
}

interface B {
  void display1();
}

class C implements A, B {
  public void display() {
    System.out.println("Hello");
  }

  public void display1() {
    System.out.println("Hi");
  }
}

public class Test {
  public static void main(String[] args) {
    C obj = new C();
    obj.display();
    obj.display1();
  }
}
```

🔷 **Output**

```
Hello
Hi
```

🧠 Easy Understanding

👉 Interface = rules
👉 Class = follows rules

🔥 One Line Difference

👉
Class = full implementation
Interface = only design

✅ Short Exam Answer

👉 Interface is a blueprint of a class that contains abstract methods.
👉 It is used to achieve abstraction and multiple inheritance.
👉 A class implements an interface using "implements" keyword and provides method body.

🧠 Memory Trick

👉
Interface = Only method name (no body)
Implements = write body

MCQ		
Sr No.	Question	Answer
1	class .....interface	implements
2	Which keyword is used to declare the interface?	interface
3	interface can have only .....methods	abstract
4	Multiple inheritance in java is achieved using	interface

✅ Q-5 Object Cloning (Simple Explanation) (Most Imp)

🔷 What is Object Cloning?

👉 Object cloning = making exact copy of an object

👉 Simple:

One object → create same duplicate object

🔷 Real Life Example

📄 Original paper → 📄 Photocopy

👉 Both look same

👉 Same in Java:

- Original object = a1
- Copy object = a2

🔷 How to Clone Object?

👉 Use clone() method

👉 This method comes from Object class

🔷 Important Rules (Very Important)

1) Must implement Cloneable interface

👉 Otherwise → error ❌

---

2) Use clone() method

👉 Defined in Object class

---

3) Call super.clone()

👉 To create actual copy

🔷 **Syntax**

public Object clone() throws CloneNotSupportedException

🔷 **Example (Fixed & Simple)**

```
class A implements Cloneable {
  int x = 10, y = 20;

  public Object clone() throws CloneNotSupportedException {
    return super.clone();
  }
}

public class Test {
  public static void main(String[] args) throws CloneNotSupportedException {

    A a1 = new A();       // original object
    A a2 = (A) a1.clone();  // cloned object

    System.out.println(a1.x);
    System.out.println(a1.y);
    System.out.println(a2.x);
    System.out.println(a2.y);
  }
}
```

🔷 **Output**

```
10
20
10
20
```

👉 **Both objects have same values** ✅

🔥 **Important Points (Exam Focus)**

- clone() method is in Object class
- Used to create copy of object
- Class must implement Cloneable
- Otherwise → CloneNotSupportedException error
- Use super.clone()

🧠 Easy Understanding

👉
a1 → original object
a2 → copy of a1

🎯 One Line Rule

👉
Object cloning = duplicate object with same data

✅ Short Exam Answer

👉 Object cloning is a process of creating an exact copy of an object.
👉 It is done using clone() method of Object class.
👉 The class must implement Cloneable interface.
👉 Otherwise, CloneNotSupportedException occurs.

MCQ		
Sr No.	Question	Answer
1	Which method is used to create exact copy of the object?	clone()
2	To use clone(), which interface class should implement?	Cloneable
3	Every class that implements clone() should call ..... to obtain the cloned object reference.	super.clone()

✅ Q-6 Nested and Inner Class (Simple Explanation)

🔷 What is Nested Class?

👉 **Nested class = class inside another class**

👉 Simple:

One class written inside another class

🔥 **Think like this (BEST WAY)**

🔷 **Real Life Example:-**

🏠 **Outer = House**

🚪 **Inner = Room inside house**

👉 Room (Inner class) **cannot exist alone**
👉 It must be inside house

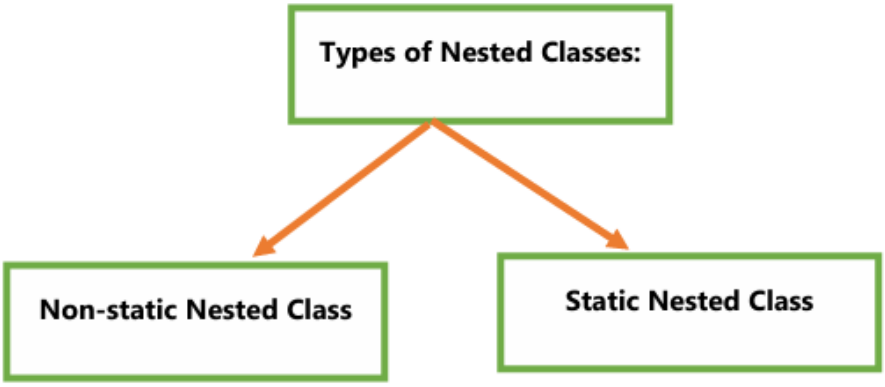
👉 Room is part of house
👉 Same in Java:

- Outer class → main class
- Inner class → inside class

🔷 Why we use it?

- To keep code **organized**
- To make code **easy to understand**
- Inner class can use outer class data

🔥 Types of Nested Class



1) Non-Static Nested Class (Inner Class)

👉 Also called **Inner Class**

🔷 Important Points

- Inside another class
- Can use outer class variables (even private) ✅
- Need to create **outer class object first**

🟢 1. INNER CLASS (Non-static)

👉 Rule:

You must create outer class first

✓ Example (Simple)

```
class Outer {
  class Inner {
    void show() {
      System.out.println("Inner class");
    }
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    Outer o = new Outer();        // create house
    Outer.Inner i = o.new Inner(); // create room

    i.show();
  }
}
```

👉 Important:

- First → Outer object
- Then → Inner object

🔵 **2. STATIC NESTED CLASS**

👉 Rule:

You can create it directly (no outer object needed)

✔ **Example**

```
class Outer {
  static class Inner {
    void show() {
      System.out.println("Static inner");
    }
  }
}

public class Test {
  public static void main(String[] args) {
    Outer.Inner i = new Outer.Inner(); // directly create

    i.show();
  }
}
```

👉 No need of Outer object ❌

🔥 **MAIN DIFFERENCE (ULTRA SIMPLE)**

👉 **INNER CLASS**

- Needs outer object
- Like **room needs house**

👉 **STATIC CLASS**

- No outer object
- Like **independent building**

🧠 **Memory Trick (Fixed)**

👉
Inner = dependent (needs outer)
Static = independent (no outer needed)

🎯 **One Line Final**

👉

- Inner → first create outer , depends on outer class
- Static → directly create , independent class

✅ Short Exam Answer

👉 Nested class is a class inside another class.
👉 It improves readability and organization.
👉 Inner class can access outer class members.
👉 There are two types: non-static (inner class) and static nested class.
👉 Inner class needs outer object, but static nested class does not.

**Difference between static and non-static nested class

Non-static nested class	Static Nested Class
1) In non-static nested class, static keyword is not used	1) In static nested class, static keyword is used
2) Non-static nested class is known as inner class	2) Static nested class is not known as inner class
3) Non-static nested class requires object of outer class	3) Static nested class does not require object of outer class
4) Non-static nested class can access the members of outer class	4) Static nested class cannot access members of the outer class

MCQ		
Sr No.	Question	Answer
1	Which keyword is used to declare static nested class?	Static
2	In ....., object of outer class is required	non-static
3	.....can access members of outer class	Non-static nested class

✅ **Q-6 Abstract Class and Final Class (Simple Explanation)**

🔵 **1. ABSTRACT CLASS**

🔷 **What is Abstract Class?**

👉 **Abstract class = incomplete class**

👉 Simple:

It has some methods without body (incomplete)

🔷 Important Points

- Declared using abstract keyword
- Can have:
  - abstract methods (no body)
  - normal methods (with body)
- Cannot create object ❌
- Must be extended (inheritance)

🔷 Example

```
abstract class Bike {
  abstract void run(); // no body

  void display() {
    System.out.println("Hello");
  }
}

class Honda extends Bike {
  public void run() {
    System.out.println("Hi");
  }
}

public class Test {
  public static void main(String[] args) {
    Honda h = new Honda();
    h.run();
    h.display();
  }
}
```

🔷 Output

```
Hi
Hello
```

🧠 **Easy Understanding**

👉 Abstract class = “rules + some ready code”
👉 Child class must complete the missing part

🔴 **2. FINAL CLASS**

🔷 **What is Final Class?**

👉 **Final class = cannot be extended**

👉 Simple:

No other class can become its child

🔷 **Important Points**

- Use final keyword
- Cannot inherit ❌
- Used for security / restriction

🔷 **Example**

```
final class Bike {
  void show() {
    System.out.println("Hello");
  }
}

class Honda extends Bike { // ❌ ERROR
}
```

👉 This gives **compile-time error**

🔷 **Final can be used in 3 ways**

1. Final variable → value cannot change

2. Final method → cannot override
3. Final class → cannot extend

🔥 MAIN DIFFERENCE (VERY IMPORTANT)

Feature	Abstract Class	Final Class
Object creation	❌ No	✅ Yes
Inheritance	✅ Must extend	❌ Cannot extend
Methods	Abstract + normal	Only normal
Purpose	Incomplete design	Restriction

🧠 Memory Trick

👉
Abstract = incomplete (needs child)
Final = locked (no child allowed)

🎯 One Line Concept

👉

- Abstract → "You MUST extend me"
- Final → "You CANNOT extend me"

✅ Short Exam Answer

👉 An abstract class is a class declared with abstract keyword that contains abstract and non-abstract methods. It cannot be instantiated and must be extended.
👉 A final class is a class declared with final keyword which cannot be inherited or extended.

MCQ		
Sr No.	Question	Answer
1	Which keyword is used to declare the class as abstract?	abstract
2	Can we create the object of abstract class?	No
3	Abstract class can contain non-abstract methods?	True
4	When final keyword is applied to the class it prevents	Inheritance

✅ Q-7 Normal Import and Static Import

🔵 1. NORMAL IMPORT

🔷 What is Normal Import?

👉 Used to **use classes of another package**

👉 Simple:

You can use class without writing full package name

🔷 Example (Without import)

```
class Test {
  public static void main(String[] args) {
    System.out.println(java.lang.Math.sqrt(25));
  }
}
```

👉 Long and not easy

🔷 Example (With Normal Import)

```
import java.lang.Math;

class Test {
  public static void main(String[] args) {
    System.out.println(Math.sqrt(25));
    System.out.println(Math.pow(2,2));
  }
}
```

👉 Easier 👍

🔑 **Key Point**

👉 You still need **class name (Math)**

🟢 **2. STATIC IMPORT**

🔷 **What is Static Import?**

👉 Used to access **static methods/variables directly**

👉 Simple:

No need to write class name

🔷 **Example**

```
import static java.lang.Math.*;

class Test {
  public static void main(String[] args) {
    System.out.println(sqrt(25));
    System.out.println(pow(2,2));
  }
}
```

👉 No Math. needed ✅

🔑 **Key Point**

👉 Direct use:

- sqrt()
- pow()

🔥 MAIN DIFFERENCE (VERY IMPORTANT)

Feature	Normal Import	Static Import
Use	Classes	Static members
Need class name	✅ Yes (Math.sqrt)	❌ No (sqrt)
Example	Math.sqrt()	sqrt()

🧠 Easy Understanding

👉
Normal import = use class
Static import = use method directly

🎯 One Line Concept

👉

- Normal → Math.sqrt()
- Static → sqrt()

✅ Short Exam Answer

👉 Normal import allows accessing classes of a package without writing full package name.
👉 Static import allows accessing static members directly without using class name.

🧠 Memory Trick

👉
Normal = Class needed
Static = No class name needed

** Difference between normal import and static import

normal import	Static import
1) The normal import provides access to class and interface	1) static import provides access to the static members of the class.
2) The normal import allows to access the class of the package without package qualification	2) Static import feature allows to access the static members of a class without class qualification.

MCQ		
Sr No.	Question	Answer
1	.....import provides access to class and interface	normal
2	.....import provides access to the static members of the class	static

✅ **Q-8 What is Package?**

🔷 **What is a Package?**

👉 **Package = collection (group) of classes and interfaces**

👉 Simple:

Package = **folder** that contains classes

🔷 **Real Life Example**

📂 Folder = Package
📄 Files = Classes

👉 Example:

- Folder: college.staff.cse
- Classes inside: Employee, Teacher

🔷 **Why we use Package?**

✓ **1. Avoid name conflict**

👉 Same class name can exist in different packages

Example:

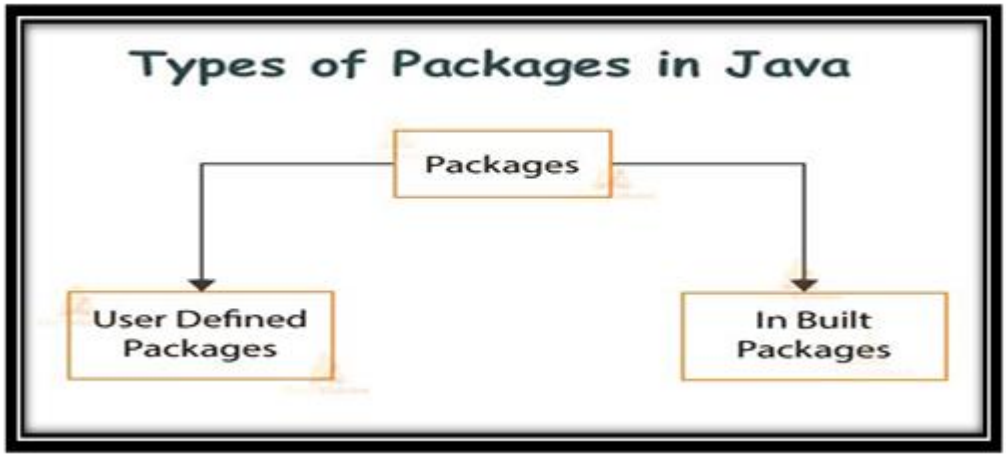
college.staff.cse.Employee
college.staff.ee.Employee

👉 No problem ✅

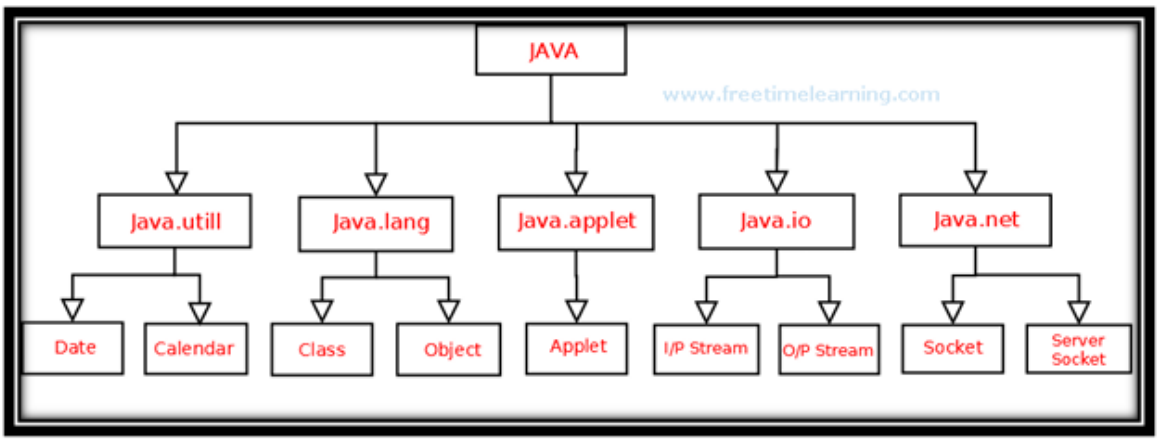
✓ **2. Better organization**

👉 Code becomes clean and easy

🔵 **Types of Packages**



❖ Following are the types of in built packages:



1) Built-in Packages

👉 Already provided by Java

🔷 **Important Built-in Packages**

1. java.lang

👉 Basic classes
👉 Automatically imported

Examples:

- String
- Math

---

2. java.io

👉 Input / Output operations

Examples:

- File handling
- Streams

---

3. java.util

👉 Utility classes

Examples:

- ArrayList
- LinkedList
- Date

---

4. java.applet

👉 Used to create applets

---

5. java.awt

👉 GUI (buttons, menus, UI)

---

6. java.net

👉 Networking

Examples:

- Internet communication

🔥 **Easy Table**

Package	Use
java.lang	Basic classes
java.io	Input / Output
java.util	Data structures
java.awt	GUI
java.net	Networking

🧠 **Memory Trick**

👉
Package = folder of classes

java. lang Package

🔷 **What is java.lang package?**

👉 It is the **most important package in Java**

👉 Simple:

It gives **basic classes needed for Java program**

👉 You don't need to import it
👉 Java **automatically imports it**

🔷 **What it provides?**

👉 It provides classes that are **basic for Java language**

Examples:

- Numbers
- Strings
- Math operations

🔥 **Important Classes in java.lang**

---

1) Math Class

👉 Used for **mathematical operations**

✓ **Example:**

```
System.out.println(Math.sqrt(25)); // 5
System.out.println(Math.pow(2,2)); // 4
```

👉 Used for:

- square root
- power

- rounding

---

2) Wrapper Classes

👉 Used to convert **primitive → object**

👉 Simple:

int → Integer
char → Character

✓ Example:

```
int a = 10;
Integer obj = a; // converting to object
```

👉 Useful in collections

---

3) String Class

👉 Used to store **text**

✓ Example:

```
String name = "Abc";
System.out.println(name.length());
```

👉 Strings are **immutable (cannot change)**

---

4) StringBuffer Class

👉 Used for **modifiable string**

👉 Simple:

String = fixed
StringBuffer = changeable

✓ Example:

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb);
```

👉 Output: Hello World

🔥 **Main Difference**

Class	Use
String	Cannot change
StringBuffer	Can change

🧠 **Memory Trick**

👉
Math → calculation
Wrapper → convert
String → text
StringBuffer → editable text

Math Class of java.lang package

🔷 **What is Math class?**

👉 Math class is in **java.lang package**
👉 Used for **mathematical calculations**

👉 Important:

All methods are **static** → use directly with class name

✓ Example:

```
Math.sqrt(25)
```

🔥 Basic Math Methods (Easy Understanding)

🔷 1. abs() → Absolute value

👉 Removes negative sign

```
Math.abs(-5) → 5
```

🔷 2. max() → Largest value

```
Math.max(10, 20) → 20
```

🔷 3. min() → Smallest value

```
Math.min(10, 20) → 10
```

🔷 4. round() → Round number

```
Math.round(2.6) → 3
Math.round(2.3) → 2
```

🔷 5. sqrt() → Square root

```
Math.sqrt(25) → 5
```

🔷 6. cbrt() → Cube root

```
Math.cbrt(27) → 3
```

🔷 7. pow() → Power

```
Math.pow(2,3) → 8
```

🔷 8. ceil() → Round UP

```
Math.ceil(2.1) → 3.0
```

🔷 9. floor() → Round DOWN

```
Math.floor(2.9) → 2.0
```

🔷 10. random() → Random number

```
Math.random() → 0.0 to 1.0
```

🔷 11. sin(), cos(), tan() → Trigonometry

```
Math.sin(0)
Math.cos(0)
Math.tan(0)
```

🔷 12. log() → Log value

```
Math.log(10)
```

🧠 Easy Memory Trick

👉

abs → no negative
max/min → compare
sqrt/cbrt → root
pow → power
ceil/floor → up/down
random → random value

🎯 One Line Concept

👉

Math class = all mathematical functions in Java

✅ Short Exam Answer

👉 The Math class in java.lang package provides various static methods for performing mathematical operations like abs(), max(), min(), sqrt(), pow(), ceil(), floor(), random(), sin(), cos(), etc.

MCQ		
Sr No.	Question	Answer
1	Math.ceil gives	Maximum round value of floating point number
2	Math.sqrt() gives.....	Square root of number
3	Math class is included in	java.lang
4	.....is the default package	java.lang

Wrapper class in java.lang package

🔷 **What is Wrapper Class?**

👉 **Wrapper class = convert primitive → object**

👉 Simple:

It wraps (covers) primitive into object

🔷 **Why needed?**

👉 Because:

- Java collections (ArrayList, etc.) use **objects only**
- Primitive (int, char) cannot be used directly

🔷 **Primitive → Wrapper Mapping**

Primitive	Wrapper
int	Integer
char	Character
boolean	Boolean
double	Double
float	Float
long	Long
short	Short
byte	Byte

🔥 1. AUTOBOXING

🔷 What is Autoboxing?

👉 **Primitive → Object automatically**

✔ Example

```
int a = 20;
Integer obj = a; // autoboxing
```

👉 int → Integer automatically ✅

🧠 Easy Meaning

👉
auto + boxing = automatic packing into object

🔥 2. UNBOXING

🔷 What is Unboxing?

👉 **Object → Primitive automatically**

✔ Example

```
Integer obj = 10;
int a = obj; // unboxing
```

👉 Integer → int automatically ✅

🧠 Easy Meaning

👉
unboxing = unpack object into primitive

🔁 Flow

👉
Primitive → Object → Primitive

🔥 Full Example

```
class Test {
  public static void main(String[] args) {

    // Autoboxing
    int a = 20;
    Integer j = a;
    System.out.println(j);

    // Unboxing
    Integer x = 30;
    int y = x;
    System.out.println(y);
  }
}
```

🧠 Memory Trick

👉
Box = put inside → Autoboxing
Unbox = take out → Unboxing

🎯 **One Line Concept**

👉
Autoboxing = primitive → object
Unboxing = object → primitive

MCQ		
Sr No.	**Question**	**Answer**
1	**.....Provides the mechanism *to convert primitive into object and object into primitive*.**	**Wrapper Class**
2	**The process of converting primitive data type into object is known as .....**	**autoboxing**
3	**The process of converting object into primitive data type is known as ....**	**unboxing**

String class in java.lang package

🔷 **What is String?**

👉 **String = sequence of characters (text)**

👉 Example:
String name = "hello";

🔷 **Important Point**

👉 Strings are **immutable**

👉 Simple:

Once created → **cannot change**

🔷 **Example**
String s = "hello"; s = "hi"; // new string created, old not changed

---

🔥 **Important String Methods**

🔷 **1. charAt()**

👉 Get character at position

```
"hello".charAt(1) → 'e'
```

🔷 **2. length()**

👉 Get string length

```
"hello".length() → 5
```

🔷 **3. substring()**

👉 Get part of string

```
"hello".substring(1) → "ello"
"hello".substring(1,3) → "el"
```

🔷 **4. contains()**

👉 Check if word exists

```
"hello".contains("el") → true
```

🔷 5. equals()

👉 Compare strings

```
"hi".equals("hi") → true
```

🔷 6. isEmpty()

👉 Check empty or not

```
"".isEmpty() → true
```

🔷 7. concat()

👉 Join strings

```
"hi".concat("hello") → "hihello"
```

🔷 8. replace()

👉 Replace characters

```
"hello".replace('l','k') → "hekko"
```

🔷 9. equalsIgnoreCase()

👉 Compare without case

```
"hi".equalsIgnoreCase("HI") → true
```

🔷 10. indexOf()

👉 Find position

```
"hello".indexOf('o') → 4
```

🔷 11. toUpperCase()

👉 Convert to uppercase

```
"hello".toUpperCase() → "HELLO"
```

🔷 12. toLowerCase()

👉 Convert to lowercase

```
"HELLO".toLowerCase() → "hello"
```

🔷 13. trim()

👉 Remove spaces

```
"  hi  ".trim() → "hi"
```

🔷 14. compareTo()

👉 Compare strings

```
"a".compareTo("a") → 0
```

🔥 Example Program (Simple)

```
class Test {
  public static void main(String[] args) {

    String s = "hello";

    System.out.println(s.charAt(1));
    System.out.println(s.length());
    System.out.println(s.substring(1));
    System.out.println(s.contains("el"));
    System.out.println(s.equals("hello"));
    System.out.println(s.toUpperCase());
  }
}
```

🧠 Easy Memory Trick

👉
charAt → char
length → size
substring → part
equals → compare
replace → change

🎯 **One Line Concept**

👉
String = text + many useful methods

MCQ		
Sr No.	**Question**	**Answer**
1	Which class contains the string functions?	**String class**
2	**substring(startind,endind) gives .....**	**Substring from start index to end index.**
3	**Which function of string class is used to merge 2 strings?**	**concat()**
4	**Which function returns the index of the given character**	**indexOf()**

StringBuffer class in java.lang package

🔷 **What is StringBuffer?**

👉 **StringBuffer = changeable string**

👉 Simple:

You can **modify (change) the text**

🔷 Difference from String

Feature	String	StringBuffer
Change value	❌ No	✅ Yes
Mutable	❌ No	✅ Yes
Performance	Slow (new object)	Fast (same object)

🧠 Easy Understanding

👉
String = fixed (cannot change)
StringBuffer = flexible (can change)

🔥 Important Features

- Can **append (add)** text
- Can **insert** text
- Can **delete / replace** text
- Automatically **increase size**

🔥 Important Methods (Very Easy)

🔷 1. append()

👉 Add text at end

"hello" → append("hi") → "hellohi"

🔷 2. insert()

👉 Add text at position

insert(2,"hi") → "hehillo"

🔷 **3. replace()**

👉 Replace part of string

replace(1,3,"ee")

🔷 **4. delete()**

👉 Remove part

delete(1,3)

🔷 **5. reverse()**

👉 Reverse string

"hello" → "olleh"

🔷 **6. capacity()**

👉 Total storage size

🔷 **7. charAt()**

👉 Get character

🔷 **8. length()**

👉 Number of characters

🔷 **9. substring()**

👉 Get part of string

🔷 **10. indexOf()**

👉 Find position

🔥 Example (Simple & Corrected)

```
class Test {
  public static void main(String[] args) {

    StringBuffer s = new StringBuffer("hello");

    System.out.println("Length: " + s.length());
    System.out.println("Insert: " + s.insert(2, "hi"));
    System.out.println("Replace: " + s.replace(1, 3, "ee"));
    System.out.println("Delete: " + s.delete(1, 3));
    System.out.println("Reverse: " + s.reverse());
    System.out.println("Capacity: " + s.capacity());
    System.out.println("CharAt: " + s.charAt(2));
    System.out.println("Substring: " + s.substring(1));
  }
}
```

🧠 Memory Trick

👉
append → add
insert → add at position
delete → remove
replace → change
reverse → reverse

🎯 One Line Concept

👉
StringBuffer = editable string

MCQ		
Sr No.	Question	Answer
1	.....is the peer class of the String class	StringBuffer
2	.....is the growable sequence of characters	StringBuffer
3	Which method is used to add insert the string in the specified string?	insert()

java. lang Package

🔷 **What is java.util package?**

👉 It is a **utility package** in Java

👉 Simple:

It gives **ready-made tools (classes)** for common tasks

🔷 **What does it provide?**

- Data structures (List, Stack, Queue)
- Date & Time
- Random numbers
- Utility functions

🔥 **Important Classes in java.util**

🔷 **1. Random**

👉 Used to generate **random numbers**

```
Random r = new Random();
System.out.println(r.nextInt(10)); // 0 to 9
```

🔷 2. Date

👉 Used to get **current date and time**

```
Date d = new Date();
System.out.println(d);
```

🔷 3. GregorianCalendar

👉 Used for **advanced date operations**

👉 Example:

- year
- month
- day

🔷 4. Vector

👉 Dynamic array (like ArrayList)

👉 Can grow automatically

🔷 5. Hashtable

👉 Stores data in **key-value pair**

```
Hashtable<Integer,String> h = new Hashtable<>();
```

🔷 6. StringTokenizer

👉 Break string into parts (tokens)

```
StringTokenizer st = new StringTokenizer("Hello World");
```

🔥 Collections in Java

👉 Collection = group of objects

🔷 Important Collections

1. LinkedList

👉 List of elements (linked)

---

2. SortedSet

👉 Stores sorted data

---

3. Stack

👉 LIFO (Last In First Out)

---

4. Queue

👉 FIFO (First In First Out)

---

5. Map

👉 Key → Value pair

🧠 Easy Memory Trick

👉
util = useful tools

🎯 One Line Concept

👉
java.util package provides **utility classes and data structures**

Random class in java.util package

🔷 **What is Random class?**

👉 Random class is in **java.util package**

👉 Used to generate:

- Random numbers
- Random values

👉 Simple:

It gives **random values every time**

🔷 **Important Point**

👉 Numbers are **pseudo-random**
👉 Means:

Looks random but generated by logic

🔥 **Common Methods (Easy Understanding)**

🔷 **1. nextInt()**

👉 Gives random integer

r.nextInt()

👉 Example output: 123, -45, 678

🔷 **2. nextInt(n)**

👉 Random number between **0 to n-1**

r.nextInt(10) → 0 to 9

🔷 **3. nextDouble()**

👉 Random decimal (0.0 to 1.0)

r.nextDouble()

🔷 **4. nextFloat()**

👉 Random float (0.0 to 1.0)

🔷 **5. nextBoolean()**

👉 True or False randomly

r.nextBoolean()

🔷 **6. nextLong()**

👉 Random long value

🔷 **7. nextBytes()**

👉 Fill byte array with random values

🔥 **Stream Methods (Advanced)**

👉 (You can just remember names)

- ints() → stream of int
- doubles() → stream of double
- longs() → stream of long

🔥 **Example (Simple & Correct)**

```
import java.util.Random;
```

```
class Test {
  public static void main(String[] args) {

    Random r = new Random();

    System.out.println("Integer: " + r.nextInt());
    System.out.println("0-9: " + r.nextInt(10));
    System.out.println("Double: " + r.nextDouble());
    System.out.println("Boolean: " + r.nextBoolean());
  }
}
```

🧠 **Easy Memory Trick**

👉
nextInt → number
nextDouble → decimal
nextBoolean → true/false

🎯 **One Line Concept**

👉
Random class = generate random values

MCQ		
Sr No.	**Question**	**Answer**
1	**Random class comes in which package?**	**Java.util**
2	**Which class generates pseudorandom numbers?**	**Random()**

Random class in java.util package

🔷 **What is Random class?**

👉 Random class is in **java.util package**

👉 Used to generate:

- Random numbers
- Random values

👉 Simple:

It gives **random values every time**

🔷 **Important Point**

👉 Numbers are **pseudo-random**

👉 Means:

Looks random but generated by logic

🔥 **Common Methods (Easy Understanding)**

🔷 **1. nextInt()**

👉 Gives random integer

r.nextInt()

👉 Example output: 123, -45, 678

🔷 **2. nextInt(n)**

👉 Random number between **0 to n-1**

r.nextInt(10) → 0 to 9

🔷 **3. nextDouble()**

👉 Random decimal (0.0 to 1.0)

r.nextDouble()

🔷 **4. nextFloat()**

👉 Random float (0.0 to 1.0)

🔷 **5. nextBoolean()**

👉 True or False randomly

r.nextBoolean()

🔷 **6. nextLong()**

👉 Random long value

🔷 **7. nextBytes()**

👉 Fill byte array with random values

🔥 **Stream Methods (Advanced)**

👉 (You can just remember names)

- ints() → stream of int
- doubles() → stream of double
- longs() → stream of long

🔥 **Example (Simple & Correct)**

```
import java.util.Random;

class Test {
  public static void main(String[] args) {
```

```
    Random r = new Random();

    System.out.println("Integer: " + r.nextInt());
    System.out.println("0-9: " + r.nextInt(10));
    System.out.println("Double: " + r.nextDouble());
    System.out.println("Boolean: " + r.nextBoolean());
  }
}
```

🧠 **Easy Memory Trick**

👉
nextInt → number
nextDouble → decimal
nextBoolean → true/false

🎯 **One Line Concept**

👉
Random class = generate random values

MCQ		
Sr No.	**Question**	**Answer**
1	**Random class comes in which package?**	**Java.util**
2	**Which class generates pseudorandom numbers?**	**Random()**

Date class in java.lang package

🔷 **What is Date class?**

👉 Date class is in **java.util package**

👉 Used to:

- Get **current date and time**
- Compare dates

👉 Simple:

It represents **date + time**

🔷 **Example (Basic)**

```
import java.util.Date;

Date d = new Date();
System.out.println(d);
```

👉 Output:

Sun Apr 05 12:30:00 IST 2026

🔥 **Important Methods (Easy Understanding)**

🔷 **1. after()**

👉 Check if date is after another

```
d1.after(d2)
```

👉 Output:

- true → after
- false → not after

🔷 **2. before()**

👉 Check if date is before another

```
d1.before(d2)
```

🔷 **3. compareTo()**

👉 Compare two dates

d1.compareTo(d2)

👉 Output:

- 0 → equal
- positive → d1 after
- negative → d1 before

🔷 **4. equals()**

👉 Check if both dates are same

d1.equals(d2)

🔷 **5. getTime()**

👉 Get time in milliseconds

d.getTime()

🔷 **6. setTime()**

👉 Change date using milliseconds

d.setTime(1000);

⚠️ **Important Correction (Your Example)**

❌ This is wrong:

Date d1 = new Date(03-12-2022);

👉 Java treats it as calculation ❌

✓ **Correct Way (Simple)**

```
Date d1 = new Date(); // current date
Date d2 = new Date();
```

👉 For custom date → use Calendar (advanced topic)

🔥 **Example (Correct & Simple)**

```
import java.util.Date;

class Test {
  public static void main(String[] args) {

    Date d1 = new Date();
    Date d2 = new Date();

    System.out.println(d1);
    System.out.println("Before: " + d1.before(d2));
    System.out.println("After: " + d1.after(d2));
    System.out.println("Compare: " + d1.compareTo(d2));
    System.out.println("Equals: " + d1.equals(d2));
    System.out.println("Time: " + d1.getTime());
  }
}
```

🧠 **Easy Memory Trick**

👉
before → earlier
after → later
compare → compare
getTime → milliseconds

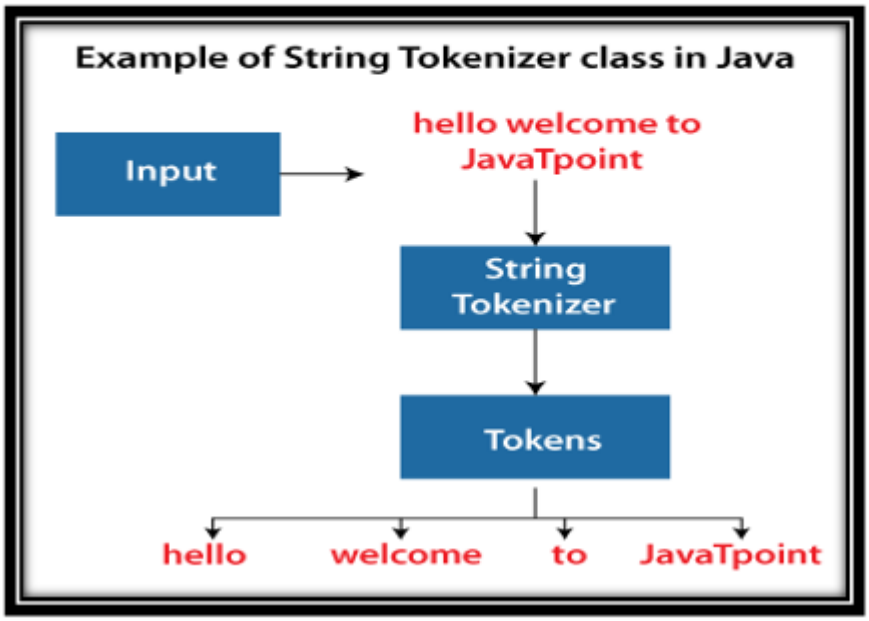
🎯 **One Line Concept**

👉
Date class = manage date and time

MCQ		
Sr No.	**Question**	**Answer**
1	**Which class of java.util package is used to give date and time**	**Date**
2	**Which method is used to check that current date is before given date or not?**	**before()**
3	**Which method of Date class is used to generate unique ID of Date object**	**hashcode**

StringTokenizer class in java.util package

- The java.util.StringTokenizer class allows you to break a string into tokens. It is simple way to break string.



🔷 **What is StringTokenizer?**

👉 StringTokenizer is in **java.util package**

👉 Used to:

Break a string into small parts (tokens)

🔷 **What is Token?**

👉 Token = **small part of string**

🔷 **Example**

"my name is karishma"

👉 Tokens:

- my
- name
- is
- karishma

🔥 **Important Methods (Easy)**

🔷 **1. countTokens()**

👉 Total number of words

st.countTokens()

🔷 **2. hasMoreTokens()**

👉 Check if more words exist

st.hasMoreTokens()

👉 Returns:

- true

- false

🔷 **3. nextToken()**

👉 Get next word

st.nextToken()

🔷 **4. nextToken(delimiter)**

👉 Split using custom symbol

new StringTokenizer("a,b,c", ",");

👉 Tokens:

- a
- b
- c

🔷 **5. hasMoreElements()**

👉 Same as hasMoreTokens()

🔷 **6. nextElement()**

👉 Same as nextToken()

🔥 **Example (Simple & Clear)**

```
import java.util.StringTokenizer;

class Test {
  public static void main(String[] args) {

    StringTokenizer st = new StringTokenizer("my name is karishma");

    System.out.println("Total Tokens: " + st.countTokens());
```

```
        while (st.hasMoreTokens()) {
            System.out.println(st.nextToken());
        }
    }
}
```

🔷 Output

Total Tokens: 4
my
name
is
karishma

🧠 Easy Memory Trick

👉
Tokenizer = split words

🎯 One Line Concept

👉
StringTokenizer = break string into words

MCQ		
Sr No.	**Question**	**Answer**
1	**Which class of java.util package allows you to break the string into tokens?**	**StringToeknizer**
2	**Which method is used to count total no. of tokens in the string?**	**countTokens()**
3	**Which method is used to print next token of the string?**	**nextToken()**

Gregorian calendar class in java.util package

🔷 **What is GregorianCalendar?**

👉 GregorianCalendar is a class in **java.util package**

👉 It is used for:

Date and time operations

🔷 **Important Idea**

👉 It is a **child class of Calendar**

🔷 **Key Difference**

🔴 **Calendar class**

- It is **abstract class**
- Cannot create object ❌

Calendar cal = Calendar.getInstance(); // only way

🟢 **GregorianCalendar class**

- It is **normal (concrete) class**
- Can create object directly ✅

GregorianCalendar g = new GregorianCalendar();

🧠 **Easy Understanding**

👉
Calendar = design (cannot use directly)
GregorianCalendar = real class (can use)

🔥 **Example (Simple)**

```
import java.util.*;

class Test {
  public static void main(String[] args) {

    Calendar cal = Calendar.getInstance();
    GregorianCalendar g = new GregorianCalendar();

    System.out.println("Calendar: " + cal.getTime());
    System.out.println("Gregorian: " + g.getTime());
  }
}
```

🔷 **Output**

👉 Both show **current date and time**

🔥 **What you can do with GregorianCalendar?**

- Get year
- Get month
- Get day
- Set date

✔ **Example**

```
GregorianCalendar g = new GregorianCalendar();

System.out.println(g.get(Calendar.YEAR));
```

```
System.out.println(g.get(Calendar.MONTH));
System.out.println(g.get(Calendar.DATE));
```

🎯 One Line Concept

👉
GregorianCalendar = class to handle date and time

🧠 Memory Trick

👉
Calendar = abstract (no object)
Gregorian = real class (object allowed)

✅ Short Exam Answer

👉 GregorianCalendar is a concrete subclass of Calendar class used to handle date and time.
👉 Calendar is abstract and cannot be instantiated, but GregorianCalendar can be instantiated directly.

MCQ		
Sr No.	Question	Answer
1	.....is the subclass of Calendar Class	Gregorian Calendar
2	We cannot create object of Calendar Class directly (T/F)?	True
3	Which method is used to create the object of calendar class?	getInstance()

Collection class in java.util package

✅ Collection Interface (Simple Explanation)

🔷 What is Collection?

👉 **Collection = group of objects**

👉 Simple:

Like a **list of items**

Example:

- List of students
- List of numbers

🔷 What is Collection Interface?

👉 It is the **main (root) interface** of collection framework

👉 Simple:

All collections come from this

🔷 Important Point

👉 You cannot create object of Collection directly ❌
👉 Java gives ready classes like:

- List
- Set
- Queue

🔥 What can Collection do?

👉 Perform operations like:

- Add data
- Delete data
- Search data
- Sort data

🔥 **Types of Collections (Very Easy)**

🔷 **1. Vector**

👉 Dynamic array (old version)

🔷 **2. Hashtable**

👉 Stores data in **key → value**

🔷 **3. LinkedList**

👉 Elements connected like chain

🔷 **4. SortedSet**

👉 Stores data in sorted order

🔷 **5. Stack**

👉 LIFO
👉 Last In First Out

Example:

- Plate stack

🔷 **6. Queue**

👉 FIFO
👉 First In First Out

Example:

- Line in ticket counter

🔷 7. Map

👉 Key → Value pair

Example:

- Roll No → Name

🧠 Easy Diagram (Understand Fast)

```
Collection (Main)
  |
  |---- List (Vector, LinkedList)
  |---- Set (SortedSet)
  |---- Queue
  |---- Map (separate)
```

🧠 Memory Trick

👉
Collection = group of data

🎯 One Line Concept

👉
Collection interface is the **root of all collection classes**

Vector class in java.util package

✅ **Vector in Java (Simple + Detailed)**

🔷 **What is Vector?**

👉 **Vector = dynamic array (resizable array)**

👉 Simple:

It can **increase or decrease size automatically**

🔷 **Why use Vector?**

👉 Because:

- No fixed size ❌
- Can store many elements ✅
- Easy to add/remove data

🔷 **Example (Real Life)**

📦 Box:

- Array → fixed size box
- Vector → expandable box

🔥 **Important Features**

- Dynamic size (grow/shrink)
- Stores objects (or generics like Integer)

- Thread-safe (important but optional for exam)

🔥 **Important Methods (Easy Explanation)**

🔷 **add()**

👉 Add element

```
ve.add(10);
```

🔷 **add(index, element)**

👉 Add at specific position

```
ve.add(1, 50);
```

🔷 **clear()**

👉 Remove all elements

1. firstElement()

👉 Returns **first element** of vector

✔ **Example:**

```
Vector<Integer> v = new Vector<>();
v.add(10);
v.add(20);

System.out.println(v.firstElement());
```

👉 Output:

```
10
```

🔷 **2. lastElement()**

👉 Returns **last element**

```
System.out.println(v.lastElement());
```

👉 Output:

20

🔷 **3. indexOf()**

👉 Returns **position (index)** of element

👉 Index starts from **0**

System.out.println(v.indexOf(20));

👉 Output:

1

👉 If element not found → returns **-1**

🔷 **4. contains()**

👉 Checks if element exists or not

System.out.println(v.contains(10));

👉 Output:

true

🔷 **5. remove()**

👉 Removes element

✔ **Two types:**

(a) Remove by index

v.remove(0);

👉 Removes first element

(b) Remove by value

v.remove(Integer.valueOf(20));

👉 Removes value 20

🔷 **6. removeElementAt()**

👉 Removes element at specific index

```
v.removeElementAt(1);
```

🔷 **7. size()**

👉 Returns number of elements

```
System.out.println(v.size());
```

👉 Output:

```
2
```

🔷 **8. capacity()**

👉 Returns **total storage space**

👉 Important:

- Default capacity = 10
- Automatically increases

```
System.out.println(v.capacity());
```

🔷 **9. clone()**

👉 Creates **copy of vector**

```
Vector<Integer> v2 = (Vector<Integer>) v.clone();
System.out.println(v2);
```

👉 Output:

```
[10, 20]
```

🔥 **Full Example (All Methods Together)**

```java
import java.util.*;

class Test {
  public static void main(String[] args) {

    Vector<Integer> v = new Vector<>();

    v.add(10);
    v.add(20);
    v.add(30);

    System.out.println("First: " + v.firstElement());
    System.out.println("Last: " + v.lastElement());
    System.out.println("Index of 20: " + v.indexOf(20));
    System.out.println("Contains 10: " + v.contains(10));
    System.out.println("Size: " + v.size());
    System.out.println("Capacity: " + v.capacity());

    v.remove(0);
    System.out.println("After remove: " + v);

    Vector<Integer> v2 = (Vector<Integer>) v.clone();
    System.out.println("Clone: " + v2);
  }
}
```

🧠 **VERY IMPORTANT CONCEPT**

👉
Index starts from 0

👉
Size ≠ Capacity

🧠 **Memory Trick (Exam)**

👉
first → first value
last → last value
indexOf → position

contains → check
remove → delete
size → count
capacity → space
clone → copy

🎯 **One Line Summary**

👉
Vector methods are used to **add, remove, search, and manage elements dynamically**

Hashtable class in java.util package

🔷 **What is Hashtable?**

👉 **Hashtable = key → value storage**

👉 Simple:

It stores data in **pairs (key, value)**

🔷 **Real Life Example**

📘 Dictionary:

- Word = Key
- Meaning = Value

👉 Same:

- Key → Value

Example:

1 → "abc"
2 → "xyz"

🔷 **Important Points**

- Each key is **unique**
- Values can be same
- Part of **java.util package**
- Works fast using hashing

🔥 **Important Methods (Detailed)**

🔷 **1. put(key, value)**

👉 Add data

h.put(1, "abc");

👉 If key exists → value is replaced

🔷 **2. remove(key)**

👉 Remove data using key

h.remove(2);

🔷 **3. size()**

👉 Total number of elements

h.size();

🔷 **4. replace(key, value)**

👉 Replace value of key

```
h.replace(1, "new");
```

🔷 5. containsKey(key)

👉 Check if key exists

```
h.containsKey(1);
```

👉 Output: true / false

🔷 6. containsValue(value)

👉 Check if value exists

```
h.containsValue("abc");
```

🔥 Your Example (Cleaned & Correct)

```
import java.util.*;

class Test {
  public static void main(String args[]) {

    Hashtable<Integer, String> h = new Hashtable<>();

    h.put(1, "abc");
    h.put(2, "xyz");

    System.out.println("Mapping: " + h);

    h.put(2, "ddd"); // replace value
    System.out.println("After replace: " + h);

    h.remove(2);
    System.out.println("After remove: " + h);

    System.out.println("Check key: " + h.containsKey(1));
    System.out.println("Check value: " + h.containsValue("abc"));
  }
}
```

🔷 Output

Mapping: {1=abc, 2=xyz}
After replace: {1=abc, 2=ddd}
After remove: {1=abc}
Check key: true
Check value: true

🔥 Important Concept

👉
Key must be unique

👉 If same key:

```
h.put(2, "xyz");
h.put(2, "ddd");
```

👉 Output:

2=ddd (old value replaced)

🧠 Memory Trick

👉
Hashtable = key-value pair

🎯 One Line Concept

👉
Hashtable stores data in **key-value format**

✅ Short Exam Answer

👉 Hashtable is a class in java.util package that stores data in key-value pairs.
👉 It provides methods like put(), remove(), size(), containsKey(), and containsValue().

Linked List class in java.util package

✅ **LinkedList in Java (Simple Explanation)**

🔷 **What is LinkedList?**

👉 **LinkedList = collection of elements (nodes)**

👉 Simple:

Elements are **connected like a chain**

🔷 **How it works?**

👉 Each element = **Node**

Each node has:

- Data (value)
- Address (next node link)

👉 So:

A → B → C → D

🔷 **Important Point**

👉 Elements are **not stored continuously (not like array)**
👉 They are connected using **links (pointers)**

🔷 **Real Life Example**

🚆 Train:

- Each coach connected to next
  👉 Same as LinkedList

🔥 Important Methods

🔷 add()

👉 Add element

s.add("A");

🔷 addFirst()

👉 Add at beginning

s.addFirst("E");

🔷 addLast()

👉 Add at end

s.addLast("D");

🔷 remove()

👉 Remove specific element

s.remove("C");

🔷 removeFirst()

👉 Remove first element

s.removeFirst();

🔷 removeLast()

👉 Remove last element

s.removeLast();

🔥 Your Example (Step-by-Step Output)

```
LinkedList<String> s = new LinkedList<>();

s.add("A");
s.add("B");
s.add("C");

s.addLast("D");   // A B C D
s.addFirst("E");  // E A B C D
```

👉 Output:

[E, A, B, C, D]

After removal:

```
s.remove("C");    // remove C
s.removeFirst();  // remove E
s.removeLast();   // remove D
```

👉 Final output:

[A, B]

🔥 Advantages

- Easy insertion/deletion ✅
- No size limit ✅

🔴 Disadvantages

- Slow access (no index direct like array) ❌

🧠 Memory Trick

👉
LinkedList = chain of nodes

🎯 One Line Concept

👉
LinkedList stores elements as **linked nodes**

✅ **Short Exam Answer**

👉 LinkedList is a class in java.util package that implements a linked list data structure.
👉 It stores elements as nodes connected by links.
👉 It provides methods like add(), addFirst(), addLast(), remove(), removeFirst(), and removeLast().

Stack class in java.util package

✅ **Stack in Java (Simple Explanation)**

🔷 **What is Stack?**

👉 **Stack = collection of elements**

👉 Works on rule:

LIFO (Last In First Out)

🔷 **Real Life Example**

📚 Stack of books:

- Last book added → removed first

👉 Same in Java Stack

🔷 **Example Flow**

Push: 10 → 11 → 12 → 13 → 14

Stack:
[10, 11, 12, 13, 14]

Top = 14

👉 Remove (pop):

[10, 11, 12, 13]

🔥 **Important Methods**

🔷 **1. push()**

👉 Add element to top

s.push(10);

🔷 **2. pop()**

👉 Remove top element

s.pop();

🔷 **3. peek()**

👉 Show top element (without removing)

s.peek();

🔷 **4. empty()**

👉 Check if stack is empty

s.empty();

👉 Output:

- true / false

🔷 **5. search()**

👉 Find position of element

s.search(11);

👉 Important:

- Counting starts from **top (1,2,3...)**

🔥 **Your Example (Step-by-Step)**

```
Stack<Integer> s = new Stack<>();

s.push(10);
s.push(11);
s.push(12);
s.push(13);
s.push(14);
```

👉 Stack:

[10, 11, 12, 13, 14]

---

pop()

s.pop();

👉 Removes 14

[10, 11, 12, 13]

---

empty()

s.empty();

👉 Output:

false

---

search()

s.search(11);

👉 Output:

3

👉 Why 3?

- From top:
  - 13 → 1
  - 12 → 2
  - 11 → 3

---

peek()

s.peek();

👉 Output:

13

🧠 **Easy Memory Trick**

👉
Stack = LIFO (Last In First Out)

🎯 **One Line Concept**

👉
Stack works on **last inserted, first removed**

✅ **Short Exam Answer**

👉 Stack is a class in Java Collection framework that follows LIFO principle.
👉 It provides methods like push(), pop(), peek(), empty(), and search().

Queue class in java.util package

✅ **Queue (Simple Explanation)**

🔷 **What is Queue?**

👉 **Queue = collection of elements**

👉 Works on rule:

FIFO (First In First Out)

🔷 **Real Life Example**

🧑‍🤝‍🧑 Line in ticket counter
👉 First person → goes first

🔷 **Simple Meaning**

👉
First added → first removed

🔥 **Important Methods (Easy)**

🔷 **add()**

👉 Add element to queue

```
p.add("abc");
```

🔷 **remove()**

👉 Remove first element (head)

```
p.remove();
```

👉 ❗ Error if queue is empty

🔷 **poll()**

👉 Remove first element safely

p.poll();

👉 No error if empty (returns null)

🔷 **peek()**

👉 Show first element (no remove)

p.peek();

🔥 **Your Code (Fixed + Clean)**

```
import java.util.*;

class Test {
  public static void main(String args[]) {

    PriorityQueue<String> p = new PriorityQueue<>();

    p.add("abc");
    p.add("xyz");
    p.add("aaa");

    System.out.println("Queue elements: " + p);

    p.remove();
    System.out.println("After remove: " + p);

    System.out.println("Poll: " + p.poll());
    System.out.println("After poll: " + p);

    System.out.println("Peek: " + p.peek());
  }
}
```

🔥 Important Note (VERY IMPORTANT)

👉 PriorityQueue does NOT follow normal order

👉 It sorts elements automatically

🔷 Example Output

Queue elements: [aaa, xyz, abc]
After remove: [abc, xyz]
Poll: abc
After poll: [xyz]
Peek: xyz

⚠️ Small Correction in Your Notes

👉 You wrote:

peek() Returns head element of the stack ❌

👉 Correct:

peek() returns **head element of queue** ✅

🧠 Memory Trick

👉
Queue = FIFO (First In First Out)

🎯 One Line Concept

👉
Queue removes element in **same order as inserted**

MCQ		
Sr No.	Question	Answer
1	.....class allows to insert and remove the elements unlike array	Vector
2	.....is the collection which represents element in key value form?	Hashtable
3	Which Class works on LIFO?	Stack
4	To create the queue, ...is used	Priority Queue

Sorted Set class in java.util package

✅ **SortedSet (Simple Explanation)**

🔷 **What is SortedSet?**

👉 **SortedSet = set of elements in sorted order**

👉 Simple:

It automatically **arranges elements**

🔷 **How sorting happens?**

👉 Two ways:

1. **Natural order** (default)
   - A → Z
   - 1 → 10
2. **Comparator** (custom order)

🔷 **Example**

Input: xyz, aaa, bbb
Output: aaa, bbb, xyz

👉 Automatically sorted ✅

🔥 **Important Methods**

🔷 **add()**

👉 Add element

s.add("abc");

🔷 **remove()**

👉 Delete element

s.remove("aaa");

🔷 **first()**

👉 First (smallest) element

s.first();

🔷 **last()**

👉 Last (largest) element

s.last();

🔷 **comparator()**

👉 Used for custom sorting (advanced)

✅ **Corrected Code**

```java
import java.util.*;

class Test {
   public static void main(String args[]) {

      SortedSet<String> s = new TreeSet<>();

      s.add("abc");
      s.add("aaa");
      s.add("xyz");
      s.add("yyy");
      s.add("bbb");

      System.out.println("Sorted Set: " + s);

      s.remove("aaa");

      System.out.println("First: " + s.first());
      System.out.println("Last: " + s.last());
   }
}
```

🔥 **Output**

```
Sorted Set: [aaa, abc, bbb, xyz, yyy]
First: abc
Last: yyy
```

🧠 **Easy Understanding**

👉
SortedSet = Set + Sorting

Map class in java.util package

- A **Map** stores data in **pairs** (one key and one value).
- Each pair is called an **entry**.
- Every **key must be unique** (no duplicate keys allowed).

👉 Example:
{1 = "Apple", 2 = "Banana"}

put()	add the elements in the Map
remove()	Removes or delete the element of Map

```
import java.util.*;

class que

{

 public static void main(String args[])

 {

  HashMap m = new HashMap();

  m.put(1, "abc");

  m.put(2, "aaa");

  System.out.println(m);

  m.remove(2);

  System.out.println(m);

 }

}
```

java.applet package

♣ This package provides classes needed to create applets and communicate with their applet.

java.awt package, java.awt.event

♣ AWT (Abstract Window Toolkit) package contains the classes for creating user interfaces and graphics. It also includes classes for painting images

♣ java.awt and java.awt.event packages are used for different events related to graphics and user interfaces

java.io package

♣ This package provides different classes for system input and output through data source, serialization and file system

♣ Different classes of java.io package are: BufferedReader, BufferedWriter, StreamReader, StreamWriter etc

java.net package

♣ This package provides the classes for implementing networking applications.

♣ It includes various classes that provide easy way to access network resource

java.swing package

♣ This package provides different swing components

♣ Swing is JFC (Java Foundation Class) and extension of AWT (Abstract Window Toolkit) and components of swing are lightweight components.

Q-8 what is User define package? How to create the package

Ans:

-> The package which is created by the user is known as User Define Package.

-> Following are the steps to create the package

Step 1: Create the notepad file

```
package mypackage;

public class Demo {

        public void display() {

          System.out.println("hi");

       }

}
```

Step 2: Create the folder named “mypackage” and save the above file with Demo.java

E:\java\mypackage\Demo.java

Step 3: Create the file and import the created package

```
import mypackage.*;
public class sample
{
        public static void main(String args[])
        {
        Demo d=new Demo();
        d.display();
        }
```

Step 4: Save the file in the root directory outside the “mypackage” folder as sample.java

E:\java\mypackage\sample.java

Step 5: Compile Package (E\java\mypackage\javac Demo.java)

Step 6: Compile the file in which you imported package (E:\java\sample.java)

Step 7: Run the File in which you imported package (E:\java\java sample)